

Breakthrough

Zastosowanie Monte-Carlo Tree Search do implementacji sztucznej inteligencji w grze Breakthrough

Adam Grącikowski i Piotr Iśtok

15 czerwca 2026

Metody Sztucznej Inteligencji 2

Plan prezentacji

Wprowadzenie

Algorytmy

Konfiguracja eksperymentów

Wyniki eksperymentów

Interfejs graficzny

Testy z udziałem ludzi

Bibliografia

Wprowadzenie

Zasady gry

Początek gry:

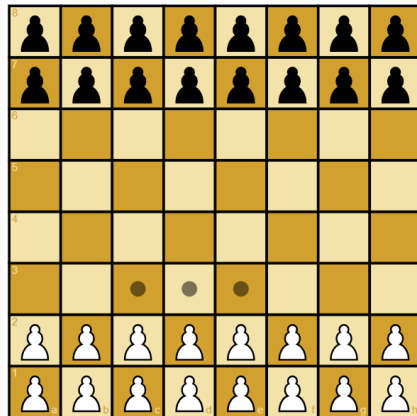
- Każdy gracz ma 16 pionów.
- Piony ustawione są na dwóch skrajnych rzędach planszy.

Ruchy:

- Ruch o jedno pole do przodu lub po skosie do przodu.
- Bicie tylko po skosie do przodu.
- Brak ruchu i bicia do tyłu oraz w poziomie.

Warunki zwycięstwa:

- Dotarcie pionem do ostatniego rzędu przeciwnika (ang. *breakthrough*).
- Zbicie wszystkich pionów przeciwnika.



Stan planszy na początku gry

Reprezentacja stanu gry

Stan planszy

Zamodelowany przy pomocy **tablic bitowych**:

- pełny układ bierek zapisany na 2 zmiennych bez znaku o rozmiarze 128 bitów (u128)
- **bit = 1** → pion na polu
- **bit = 0** → puste pole

Kontekst rozgrywki

- znacznik aktualnego gracza
- brak dodatkowych flag (jak w szachach)

Rozmiar struktury: 33 bajty

Zalety

- wysoka wydajność operacji bitowych (AND/OR/XOR)
- mały rozmiar i narzut pamięciowy
- masowe przetwarzanie stanów

Ograniczenia

- pojemność do maksymalnie 128 pól
- w praktyce plansza do 11×11

Algorytmy

- **Minimax** (agent heurystyczny)
- **MCTS** [1, 2]
- **MCTS + RAVE** [6, 7]
- **MCTS + Heavy Playouts** [4]

Przeszukiwanie drzewa gry

- Zastosowanie klasycznego algorytmu **Minimax z odcięciami Alfa-Beta** (ang. *Alpha-Beta pruning*).
- Ograniczenie głębokości i deterministyczne działanie w oparciu o funkcję oceny pozycji E .

Kryteria oceny pozycji

- Zaawansowanie pionów.
- Przewaga materialna.
- Struktury obronne.
- Kara za grę kolumnami A i H (piony mają tam tylko dwie opcje ruchu zamiast trzech).

Proces decyzyjny

Podobnie jak ludzie planuje swoje akcje kilka kroków w przód, a następnie intuicyjnie ocenia ($\max\{E(s_t) : s_t \in S_t\}$), czy powstała w ten sposób sytuacja na planszy jest dla niego korzystna.

Idea

- Probabilistyczne przeszukiwanie drzewa gry [1].
- Drzewo rozwijane asymetrycznie – więcej iteracji trafia do obiecujących wariantów.
- Węzeł przechowuje stan planszy, gracza oraz statystyki odwiedzin i zwycięstw.

Jedna iteracja

1. Selekcja
2. Ekspansja
3. Symulacja
4. Propagacja wsteczna

Selekcja UCB1 [2]

$$\frac{w_i}{n_i} + C \sqrt{\frac{\ln N}{n_i}}$$

Eksploracja dobrych ruchów jest równoważona eksploracją rzadziej odwiedzonych gałęzi.

AMAF [6]

- RAVE przyspiesza uczenie statystyk na początku przeszukiwania [6].
- Ruch zagrany później w symulacji traktujemy tak, jakby mógł być wartościowy także wcześniej.
- Oprócz statystyk MCTS przechowujemy statystyki AMAF: $\tilde{w}_i$, $\tilde{n}_i$.

Połączenie ocen

$$V_{RAVE} = (1 - \beta) \frac{w_i}{n_i} + \beta \frac{\tilde{w}_i}{\tilde{n}_i}$$

$$\beta = \frac{k}{k + n_i}$$

Gdy węzeł ma mało odwiedzin, większą rolę ma szybka estymacja AMAF. Wraz ze wzrostem n_i algorytm przechodzi do klasycznej oceny UCT.

Problem klasycznego rollout'u

- Standardowy MCTS symuluje partię losowymi ruchami.
- W Breakthrough losowe symulacje często tworzą mało realistyczne przebiegi.
- Heavy Playouts kierują symulację wiedzą dziedzinową [4].

Polityka ϵ -greedy [7]

- z prawdopodobieństwem ϵ ruch losowy,
- z prawdopodobieństwem $1 - \epsilon$ najlepszy ruch według E .

Funkcja oceny

W symulacjach wykorzystywana jest ta sama funkcja heurystyczna E , co w agencji Minimax: przewaga materialna, zaawansowanie pionów, struktury obronne oraz kara za grę krawędzią.

Cena tej modyfikacji to większy koszt obliczeniowy każdej symulacji.

Konfiguracja eksperymentów

Format pliku wejściowego

```
board_width = 6
board_height = 7
seed = 100

[white_player]
type = "Minimax"
max_depth = 5
material_weight = 250
advancement_weight = 10

[black_player]
type = "Mcts"
max_iterations = 100000
exploration_constant = 1.41
use_heavy_playouts = true
```

Przykładowy plik konfiguracyjny.

Charakterystyka:

- Konfiguracja graczy w formacie TOML.
- Brakujące pola uzupełniane wartościami domyślnymi.

Tabela 1: Pola konfiguracyjne dla agenta typu Minimax.

Pole	Domyślnie	Opis
max_depth	4	Maksymalna głębokość przeszukiwania drzewa gry.
material_weight	200	Waga przewagi materialnej (liczby bierok).
advancement_weight	5	Premia za zaawansowanie pionów w kierunku celu przeciwnika.
defended_weight	5	Premia za wzajemnie broniące się piony.
edge_penalty_weight	-2	Kara za ustawianie pionów na krawędziach planszy.

Tabela 2: Pola konfiguracyjne dla agenta typu MCTS.

Pole	Domyślnie	Opis
max_iterations	75000	Maksymalna liczba iteracji algorytmu MCTS.
max_time_ms	None	Limit czasu na ruch (ms). Nadpisuje liczbę iteracji.
exploration_constant	1.41	Stała eksploracji (C) w formule UCT.
use_rave	false	Aktywuje modyfikację RAVE.
rave_k	1000.0	Waga heurystyki RAVE w stosunku do UCT.
use_heavy_playouts	false	Aktywuje symulacje kierowane heurystyką.
heavy_playouts_epsilon	0.1	Szansa na losowy ruch (ϵ) w ulepszonych symulacjach.
material_weight	200	Waga przewagi materialnej.
advancement_weight	5	Waga zaawansowania pionów.
defended_weight	5	Premia za bronione piony.
edge_penalty_weight	-2	Kara za piony na krawędzi.

Wyniki eksperymentów

Przewaga wariantów MCTS nad Minimax

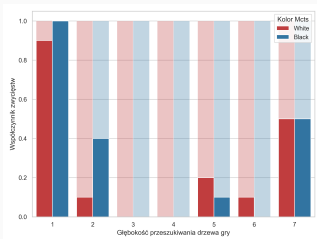
Metodologia:

- 3 warianty: MCTS, MCTS + RAVE, MCTS + Heavy Playouts
- Oponent: Minimax z głębokością $d \in [1, 7]$
- 20 partii na każdą głębokość (10 białe, 10 czarne)
- Łącznie: 420 gier
- Stały budżet MCTS: 10000 iteracji

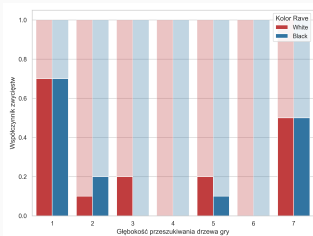
Metryki:

- Współczynnik zwycięstw
- Średnia długość gry
- Czas decyzji

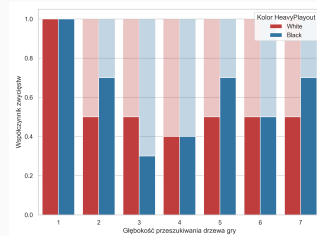
Współczynnik zwycięstw



MCTS



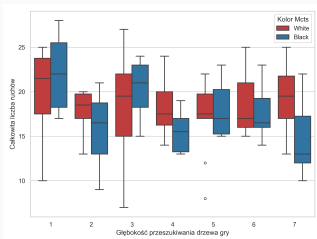
MCTS + RAVE



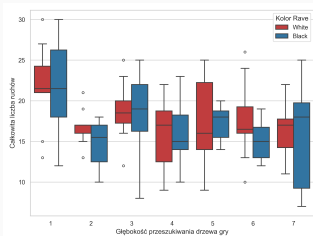
MCTS + Heavy Playouts

Porównanie odsetka wygranych partii w zależności od głębokości przeszukiwania agenta Minimax dla różnych wariantów algorytmu MCTS.

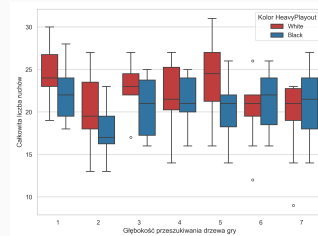
Całkowita liczba ruchów



MCTS



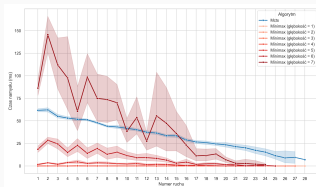
MCTS + RAVE



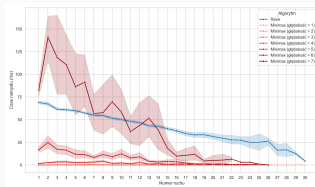
MCTS + Heavy Playouts

Zestawienie dystrybucji całkowitej liczby ruchów rozegranych w partiach przeciwko agentowi Minimax dla różnych wariantów algorytmu MCTS.

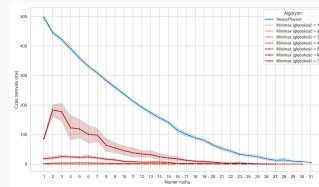
Średni czas namysłu



MCTS



MCTS + RAVE



MCTS + Heavy Playouts

Średni czas namysłu (w milisekundach) dla różnych wariantów algorytmu MCTS oraz ich przeciwników (agenta Minimax o rosnącej głębokości przeszukiwania).

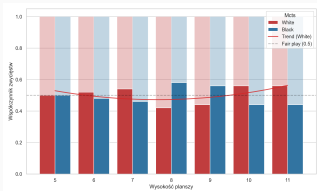
Metodologia

- 3 warianty: MCTS, MCTS + RAVE, MCTS + Heavy Playouts
- Parametry planszy:
 - stała szerokość $W = 8$
 - zmienna wysokość $H \in \{5, 6, 7, 8, 9, 10, 11\}$
- Budżet obliczeniowy: $I = 2000 \cdot H$
- Identyeczni agenci po obu stronach planszy
- 50 partii dla każdego rozmiaru planszy

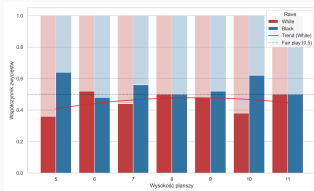
Metryki:

- Współczynnik zwycięstw dla białych

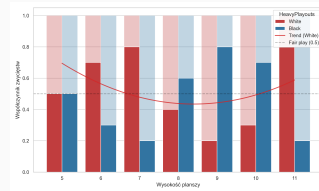
Średni czas namysłu



MCTS



MCTS + RAVE



MCTS + Heavy Playouts

Odsetek zwycięstw gracza rozpoczynającego (Białe) w starciach symetrycznych w zależności od wysokości planszy (H) przy stałej szerokości $W = 8$.

Wpływ modyfikacji RAVE na MCTS – metodologia

Cel eksperymentu

- Bezpośrednie porównanie MCTS + RAVE z klasycznym MCTS.
- Kryterium sukcesu: ponad 55% zwycięstw agenta RAVE.
- Stały budżet: 10 000 iteracji na ruch dla obu agentów.

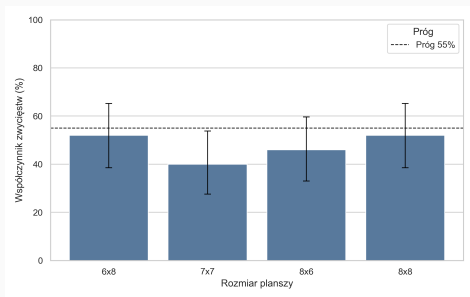
Dobór parametru k

- Testowano wpływ parametru k na wagę AMAF.
- Wybrano $k = 100$, bo dawało lepsze wyniki niż domyślne $k = 1000$.
- Mniejsze k szybciej przełącza algorytm z RAVE na klasyczne statystyki UCT.

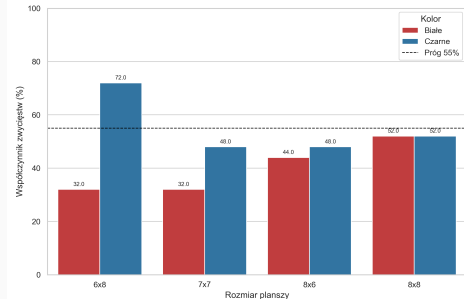
Zakres testów

- Plansze: 6×8 , 7×7 , 8×6 , 8×8 .
- 50 partii na każdy rozmiar planszy: 25 jako białe i 25 jako czarne.
- Łącznie 200 partii.

Wpływ modyfikacji RAVE na MCTS – wyniki



Wyniki według rozmiaru planszy



Wyniki z podziałem na kolor

- MCTS + RAVE wygrał 95 z 200 partii, czyli 47.5%.
- W żadnym rozmiarze planszy wynik nie przekroczył progu 55%.
- Hipoteza H1 została odrzucona: AMAF nie poprawił stabilnie decyzji względem klasycznego MCTS.

Wpływ Heavy Playouts na MCTS – metodologia

Cel eksperymentu

- Bezpośrednie porównanie MCTS + Heavy Playouts z klasycznym MCTS.
- Kryterium sukcesu: ponad 55% zwycięstw wariantu heurystycznego.
- Heavy Playouts używa droższej, ale bardziej informacyjnej symulacji.

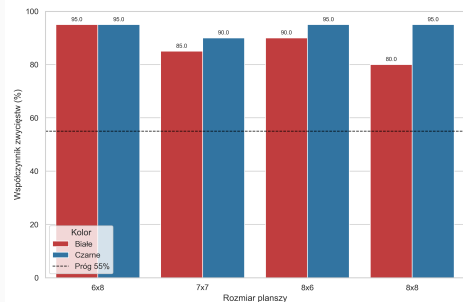
Limit czasu

- Zastosowano limit 250 ms na ruch.
- Obaj agenci mieli taki sam budżet czasowy.
- Limit czasu jest ważniejszy niż stała liczba iteracji, bo pojedynczy playout jest wolniejszy.

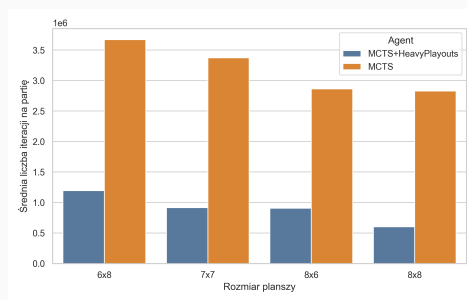
Zakres testów

- Plansze: 6×8 , 7×7 , 8×6 , 8×8 .
- 40 partii na każdy rozmiar planszy: 20 jako białe i 20 jako czarne.
- Strategia symulacji: ϵ -greedy z $\epsilon = 0.1$.

Wpływ Heavy Playouts na MCTS – wyniki



Skuteczność z podziałem na kolor



Liczba iteracji w limicie 250 ms

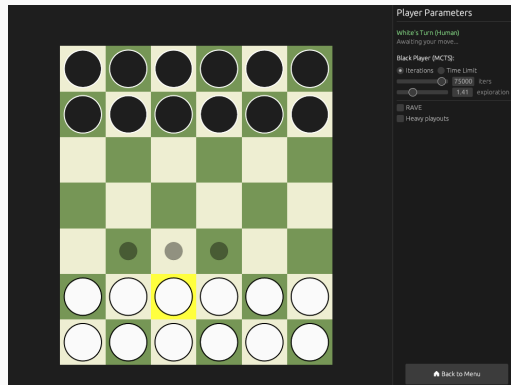
- Heavy Playouts wygrał 145 ze 160 partii, czyli 90.6%.
- Agent wykonywał mniej iteracji niż klasyczny MCTS, ale każda symulacja dawała lepszą informację.
- Hipoteza H2 została przyjęta: wiedza dziedzinowa w playoutach wyraźnie poprawiała decyzje.

Interfejs graficzny

Menu i rozgrywka

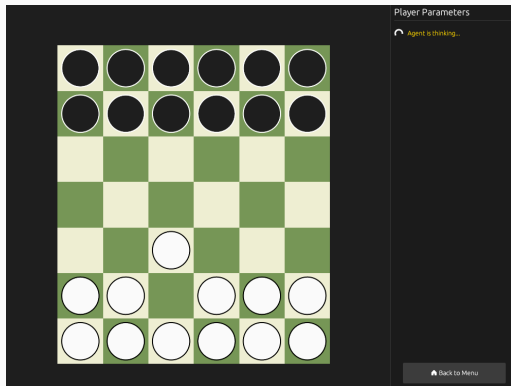


Ekran startowy

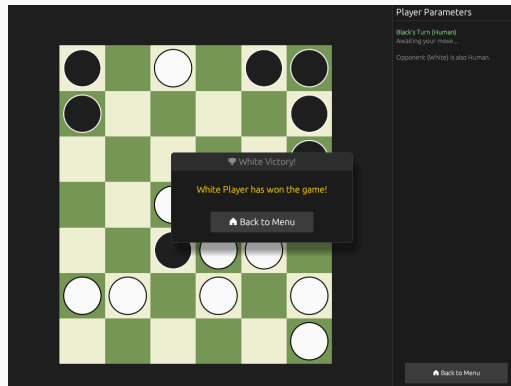


Oczekiwanie na ruch gracza

Ruch agenta i koniec gry



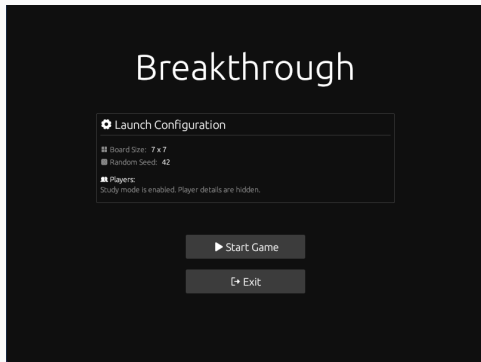
Oczekiwanie na ruch agenta



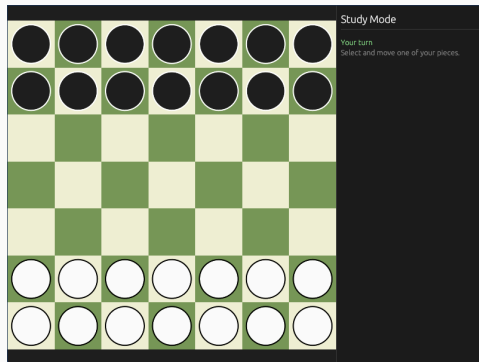
Ekran końcowy

Testy z udziałem ludzi

Tryb ślepego testu



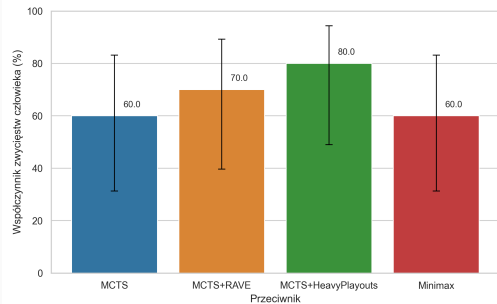
Ekran startowy bez nazwy agenta



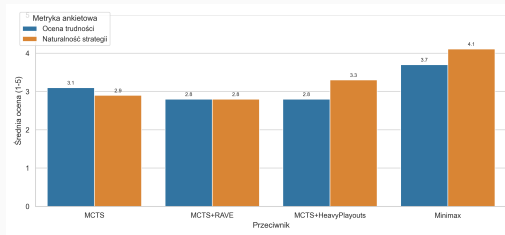
Rozgrywka z ukrytym typem przeciwnika

- Badani nie widzieli, czy grają przeciwko Minimax, MCTS, RAVE czy Heavy Playouts.
- Celem było oddzielenie realnej skuteczności agenta od sugestii wynikającej z jego nazwy.

Wyniki testów z udziałem ludzi



Współczynnik zwycięstw ludzi



Ocena trudności i naturalności strategii

- Agenty grały sensownie, ale w testach nadal były słabsze od ludzi.
- Minimax był odbierany jako najbardziej naturalny i najtrudniejszy przeciwnik.

Bibliografia

-  R. Coulom, *Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search*, w: *Computers and Games*, Springer, Berlin, Heidelberg, 2006, s. 72–83.
-  L. Kocsis, C. Szepesvári, *Bandit Based Monte-Carlo Planning*, w: *Machine Learning: ECML 2006*, Springer, Berlin, Heidelberg, 2006, s. 282–293.
-  A. Saffidine, N. Jouandea, and T. Cazenave, *Solving Breakthrough with Race Patterns and Job-Level Proof Number Search*, w: *Advances in Computer Games*, Springer, Berlin, Heidelberg, 2012, s. 236–247.
-  C. B. Browne i in., *A Survey of Monte Carlo Tree Search Methods*, w: *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, 2012, s. 1–43.
-  S. J. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed., Prentice Hall, 2010.

-  S. Gelly, D. Silver, *Monte-Carlo tree search and rapid action value estimation in computer Go*, w: *Proceedings of the 24th international conference on Machine learning (ICML '07)*, ACM, New York, 2007, s. 273–280.
-  R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, Cambridge, MA, 2018.
-  D. E. Knuth, R. W. Moore, *An Analysis of Alpha-Beta Pruning*, w: *Artificial Intelligence*, vol. 6, no. 4, 1975, s. 293–326.
-  Klabnik, S., & Nichols, C. (2023). *The Rust Programming Language*. No Starch Press.
Dostępne pod adresem: <https://www.rust-lang.org/>
-  Python Software Foundation (2026). *Python 3 Reference Manual*.
Dostępne pod adresem: <https://www.python.org/>

Dziękujemy za uwagę!